

Modules

A python module can be defined as a python program file which contains a python code including python functions, class, or variables. In other words, we can say that our python code file saved with the extension (.py) is treated as the module. We may have a runnable code inside the python module. Modules in Python provides us the flexibility to organize the code in a logical way.

To use the functionality of one module into another, we must have to import the specific module.

Let's create the module named as file.py.

```
#displayMsg prints a message to the name being passed.  
def displayMsg(name):  
    print("Hi "+name)  
def show():  
    print("show here")
```

Loading the module in our python code

We need to load the module in our python code to use its functionality. Python provides two types of statements as defined below.

1. import statement
2. from-import statement

import statement

The import statement is used to import all the functionality of one module into another. Here, we must notice that we can use the functionality of any python source file by importing that file as the module into another python source file.

We can import multiple modules with a single import statement, but a module is loaded once regardless of the number of times, it has been imported into our file.

The syntax to use the import statement is given below.

```
import module1,module2,..... module n
```

Hence, if we need to call the function displayMsg() defined in the file file.py, we have to import that file as a module into our module as shown in the example below.

```
import file
name = input("Enter the name:")
file.displayMsg(name)
file.show()
```

output:

Enter the name:kochi

Hi kochi

show here

from-import statement

Instead of importing the whole module into the namespace, python provides the flexibility to import only the specific attributes of a module. This can be done by using from? import statement.

The syntax to use the from-import statement is given below.

```
from < module-name> import <name 1>, <name 2>..,<name n>
```

Consider the following module named as calculation which contains three functions as summation, multiplication, and divide.

Create a file calculation.py and add the following code

```
#place the code in the calculation.py
def summation(a,b):
    return a+b
def multiplication(a,b):
    return a*b
def divide(a,b):
    return a/b
```

Create a file main.py and add the following code

```
from calculation import summation
#it will import only the summation() from calculation.py
myvar1 = int(input("Enter the first number"))
myvar2 = int(input("Enter the second number"))
print("Sum = ",summation(myvar1,myvar2)) #we do not need to
specify the module name while accessing summation()
```

output:

*Enter the first number*10

*Enter the second number*20

Sum = 30

The from...import statement is always better to use if we know the attributes to be imported from the module in advance. It doesn't let our code to be heavier. We can also import all the attributes from a module by using *.

Consider the following syntax.

```
from <module> import *
```

Example

```
from calculation import *
```

This imports all functions defined in the calculations.py

Built in Modules

Python comes with a rich set of **built-in modules** that provide standardized solutions for many problems, including file I/O, system calls, data serialization, math operations, and more. Some of them are mentioned below.

1. Datetime
2. Time
3. Math

4. Statistics
5. Random

Datetime module

A date in Python is not a data type of its own, but we can import a module named datetime to work with dates as date objects.

Import the datetime module and display the current date:

```
import datetime
x = datetime.datetime.now()
print(x)
```

output:

2023-02-11 16:42:18.230886

Date Output

The date contains year, month, day, hour, minute, second, and microsecond.

The datetime module has many methods to return information about the date object. Return the year and name of weekday:

Creating Date Objects

To create a date, we can use the datetime() class (constructor) of the datetime module. The datetime() class requires three parameters to create a date: year, month, day.

```
import datetime
x = datetime.datetime(2025, 1, 1)
print(x)
```

output:

2020-01-01 00:00:00

The `datetime()` class also takes parameters `fortime` and `timezone` (hour, minute, second, microsecond, tzzone), but they are optional, and has a default value of 0

strftime() method

The `datetime` object has a method for formatting date objects into readable strings. The method is called `strftime()`, and takes one parameter, `format`, to specify the format of the returned string

Legal format codes:

%a Weekday, short version

```
import datetime
x = datetime.datetime.now()
print(x.strftime("%a"))
```

output:

Tue

%A Weekday, full version

```
import datetime
x = datetime.datetime.now()
print(x.strftime("%A"))
```

output:

Tuesday

%w Weekday as a number 0-6, 0 is Sunday

```
import datetime
x = datetime.datetime.now()
print(x.strftime("%w"))
```

output:

2

%d Day of month 01-31 30

%b Month name, short version Dec

%B Month name, full version October

%m Month as a number 01-12 12

%y Year, short version, without century 25

%Y Year, full version 2024

%H Hour 00-23 10

%I Hour 00-12 05

%p AM/PM PM

%M Minute 00-59 41

%S Second 00-59 08

%f Microsecond 000000-999999 548513

%j Day number of year 001-366 365

%W Week number of year, Monday as the first day of week,
00-53 52

Time module

The time module in Python is part of the standard library and provides various time-related functions, including getting the current time, pausing execution, formatting and parsing time strings, and measuring performance. Your Python code gets the current time in seconds since the **Unix epoch** (January 1, 1970) using `time.time()` and prints it.

1. `time.time()` – Get current time in seconds

```
import time
current_time = time.time()
print("Current timestamp:", current_time)
```

output:

Current timestamp: 1750307126.5509207

2. Getting time string from seconds

The code uses the time module to convert a specified timestamp (1750307126.5509207) into a human-readable date and time format.

```
import time
curr = time.ctime(1750307194.392421)
print("Current time:", curr)
```

output:

Current time: Thu Jun 19 09:56:34 2025

3. `time.sleep(seconds)` – Pause execution for a given number of seconds

```
import time
time.sleep(10)
print("Hi")#prints after 10 seconds
```

output:

Hi



smectechnologies
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215
GUIDING THE FUTURE WITH AI